Deuteron

The compiler transformations we built

Scheduled Emmy IR → PTX · RTX 5090

Recognize structure. Check the rewrite. Change the GPU work.

Reuse data

Vectorize loads
Keep reduction inputs

Redistribute work

Cooperate across lanes
Fold eligible K partitions

Build pipelines

Pack tensor operands
Stage attention weights

Where our passes run

Our contribution: structural recognizers, rewrite guards, and transformed GPU schedules.

01

Validate

Scheduled IR
+ launch metadata
Types and contract

→

02

Rewrite

Vectorize → forward loads
Reorder → fold split-K

→

03

Lower

Reductions
Tensor pipelines
Staged recurrences

→

04

LLVM → PTX

Typed LLVM IR
LLVM optimization
NVPTX code generation

Actual early rewrite order

`vectorize.py` → `regforward.py` → `recurrence.py` → `gridfold.py`

Each rewrite checks its own indices, ownership, and lifetime conditions.

Reuse data before reloading it

Elementwise vectorization · Reduction load forwarding · Coalesced sum/square reduction

Scalar loop

load → load → load → load

Four contiguous values per lane

x0 | x1 | x2 | x3

Reduction load forwarding

load vector → reduce → consumer

Reuse the same registers; skip the reload.

Coalesced sum/square reduction

Contiguous load batches

Long loops: independent accumulators

Pass-specific guards: matching indices · readonly inputs · closed lifetimes · complete coverage

Reshape parallel work

Cooperative dot reduction · Small-tree elimination · Split-K grid folding

Lane / warp work

Repeated dot → lane partials → shuffle

p0 p1 p2 p3

Closed small trees → direct sums

Warp 0 can combine up to 32 partials.

Split-K grid folding

Partitions per output tile in a larger grid

CTA 0 | CTA 1 | exit | exit

Logical K partitions: 4 → 2

K work / active CTA: 1× → 2×

Same physical grid. Same atomic output.

Illustrative factor 2. Actual folding checks coverage, scratch capacity, K depth, and remaining parallelism.

Change who does the work while retaining the original launch contract.

Recover the contraction

Contraction tensorization · Tensor shared-memory layout

Shared multiply + add

Recognize a closed contraction

Prove affine tile coverage

Derive full-warp ownership

Guarded substitution → warp MMA

→

BF16x3 or TF32x3

$A \approx Ah + Al \qquad B \approx Bh + Bl$

$$Ah \cdot Bh + Ah \cdot Bl + Al \cdot Bh$$

FP32 buffers + accumulation

Low × low omitted → approximate products

Ordinary FP32 contraction mode is also available.

SEPARATE FP16 STORAGE

Pointwise storage and conversion

FP16 → explicit FP32 arithmetic → FP16

Exact widening · nearest-even narrowing

Feed tensor cores, then drain

Packed register staging · Tensor fragment pipelining · Pipeline drain

SCHEDULED / NONPACKED

FP32 cp.async ring → shared FP32 → convert → MMA

Certified packed BF16 path

Vector global prefetch → Pack high + residual → One used shared stage → Load next fragment → Current MMA

Tile dataflow with global and fragment lookahead.

Innovation: certify the exact copy phase and the packed producer / consumer layout.

Final tile: omit unused prefetch + consumer barrier; keep producer writes and their barrier.

Eligible FP32 / TF32x3 paths retain 4 / 8 / 16-byte cp.async groups.

Stage the attention recurrence

Staged online recurrence · Blocked recurrence loads · Recurrence shared-storage reuse

BEFORE Output columns repeat score and online max / mass / value work

Scores once
per sharing group

→

Maximum
across the keys

→

Weights
exp(score − max)

→ **Register mass**

→ **Shared weights → PV**

Unnormalized weights; normalization stays in the output suffix.

Blocked recurrence loads

2 / 4 adjacent keys per lane
Vector loads and hoisted addresses

Recurrence shared-storage reuse

Same warp + same component
Read → maximum → overwrite

Redistribute probability–value work

Pass: Warp-distributed vector PV reduction

BEFORE

1 lane

1 column × every key

```
sum_k p[k] * V[k,c]
```

Guard: 1 warp · 32 columns
Aligned loads + closed suffix

AFTER · 4 groups × 8 lanes

Four adjacent columns per lane

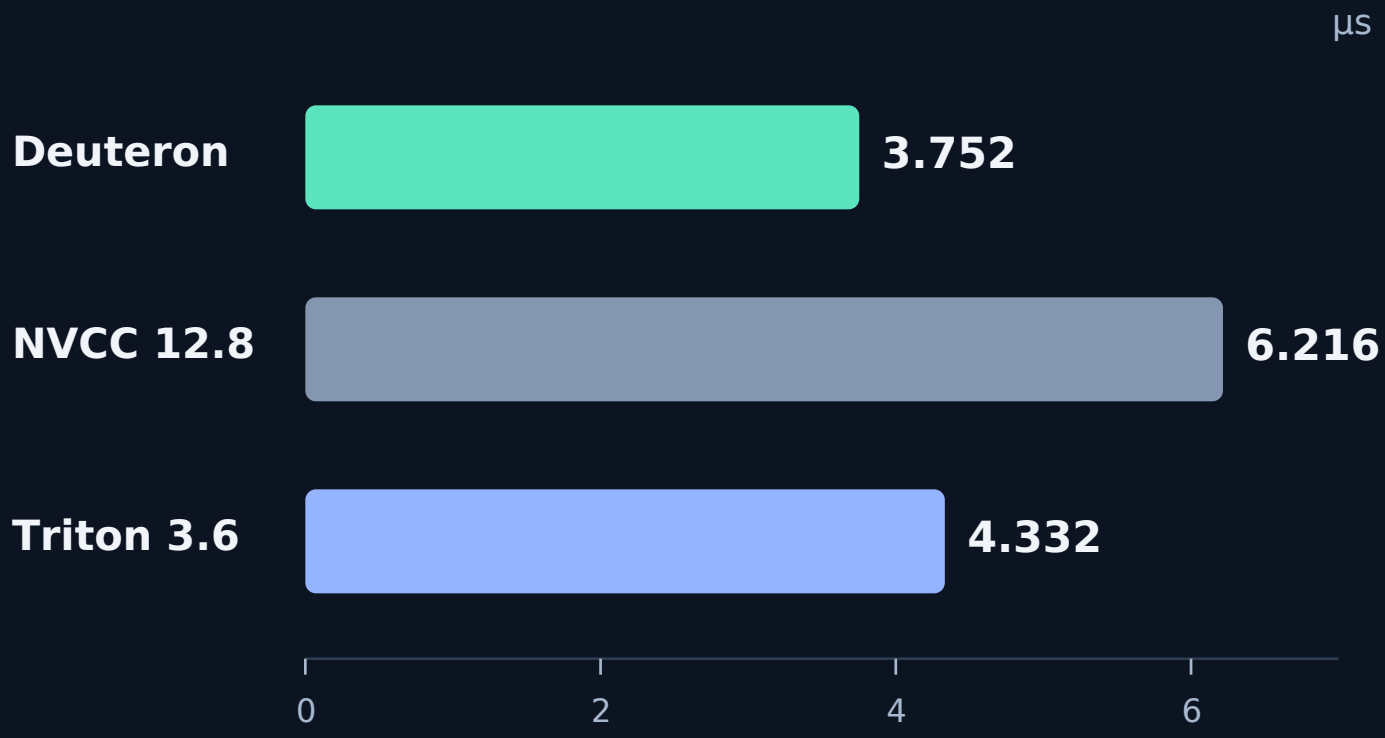
Shard 0	0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31
Shard 1	0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31
Shard 2	0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31
Shard 3	0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31

Shuffle offsets 16 + 8 → first 8 lanes write

Correct suffix replay: restore the saved denominator before every column.

RTX 5090 · whole-compiler results

Geometric mean warm latency · 15 open workloads · lower is better



1.66×

vs generated CUDA / NVCC

1.15×

vs tested Triton baseline

270 balanced warm calls · Original inputs and numerical tolerances · All output checks pass

20 warmups / 200 iterations · Limited Triton search · Compile / assembly / transport excluded